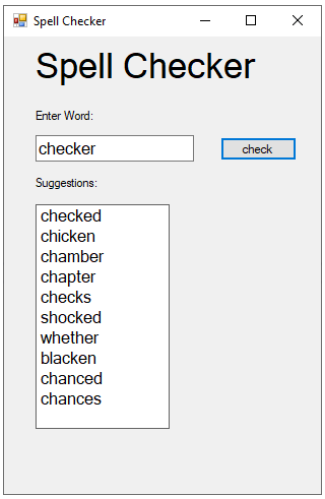
CS 340 — Project 4

Spell Checker with Sequence Alignment

Worth: 150 points **Submit:** your GitHub Pages URL and repository link on Blackboard

Goal



You will build a spell checker, like the one in a word processor: a user types a word, and the program suggests the words it most likely meant. Your checker compares the typed word against a dictionary using the dynamic-programming sequence-alignment algorithm from the lectures, and displays the ten closest matches. You will then deploy it as a live web page so anyone can use it from a browser.

What you are given

- **dictionary.js** — 6,995 words drawn from the novel Pride and Prejudice, provided as a global JavaScript array named DICTIONARY. Include it with a <script> tag and use it directly.
- **index.html** — a starter page with the text box, button, results area, and all the browser wiring already written. Two functions are left as stubs for you to complete: alignmentScore and getSuggestions.

Language: this project is written in JavaScript. Unlike the earlier projects, you do not choose the language — a single shared language is what lets everyone deploy to the web the same way. The algorithm itself is just arrays and loops, so the JavaScript you need is minimal.

Background: the alignment penalty

Sequence alignment lines up two words and charges a penalty for every place they differ. The smaller the total penalty, the more alike the two words are — so the dictionary words with the smallest penalties are the best spelling suggestions. Use these penalties:

Situation	Penalty
Exact match (the two letters are the same)	0
Consonant/consonant or vowel/vowel mismatch	1
Vowel/consonant mismatch	3
Gap (insert or delete one letter)	2

Part 1 — Implement the alignment (alignmentScore)

Complete the alignmentScore(word1, word2) function so that it returns the minimum total penalty for aligning the two words, computed with the dynamic-programming table from the lectures (build an (n+1) by (m+1) grid, fill the first row and column with gap penalties, then fill each remaining cell from the cell above, the cell to the left, and the diagonal cell). The value in the final cell is the answer.

Running time. Each call to alignmentScore must run in O(n × m) time, where n and m are the lengths of the two words — that is, the cost of filling the table once. Your spell checker will call

this function once for every dictionary word, so a single query performs about 6,995 alignments. That is still fast enough to feel instant in a browser, but make sure one alignment is O(n × m) and not worse.

Part 2 — Find the ten best suggestions (getSuggestions)

Complete getSuggestions(typedWord) so that it compares the typed word against every word in DICTIONARY using alignmentScore, then returns the ten entries with the lowest scores. Return each as an object { word, score }, sorted from the lowest score to the highest. Many words will tie at the same score, so break ties alphabetically.

Part 3 — Run the web app

Open index.html in a browser. Once your two functions are written, typing a word and clicking Check should list ten suggestions with their scores. The page and all of its wiring are already provided — your job is to make the two functions work so the page does something useful.

Part 4 — Deploy to the web with GitHub Pages

Publish your working app so it runs from a public URL:

1. Create a free GitHub account if you do not have one, and make a new public repository.
2. Add your index.html and dictionary.js files to the repository (index.html must be at the top level).
3. In the repository, go to Settings > Pages, and under “Build and deployment” choose to deploy from your main branch. Save.
4. Wait a minute, then visit the URL GitHub gives you (it looks like https://yourname.github.io/your-repo/). Confirm the spell checker works there.

Step-by-step instructions are at https://pages.github.com.

If deployment gives you trouble: don’t let it sink the whole project. You may instead drag-and-drop your files onto a free host such as netlify.com/drop, or, as a last resort, submit your index.html and dictionary.js along with a short screen recording of the working app for partial credit on the deployment portion.

Verify your work

Two quick checks before you submit:

- **Algorithm:** alignmentScore("wills", "waltz") must return 3. (This is the same matrix from the sequence-alignment quiz — work it out by hand if you want to confirm the whole table.)
- **Suggestions:** typing “freind” should put “friend” at the top; “beautyful” should suggest “beautiful”; and “elizibeth” should suggest “elizabeth.”

Note that many words tie at the same low score (for example, several words tie for a typo like “wether”), which is why your tie-breaking rule matters for which ten appear.

Submission

On Blackboard, submit (1) the live URL where your spell checker runs and (2) a link to your GitHub repository. Due at the end of Week 8; the standard two-day / 20% late policy applies.

Grading rubric (150 points)

Component	Points	What earns full marks
Sequence alignment algorithm	80	alignmentScore is a correct O(n x m) dynamic program with the given penalties. Partial credit: correct table borders; correct recurrence; correct minimum returned.
Finding the ten best suggestions	20	Scores every dictionary word and returns the ten lowest, sorted, with ties broken alphabetically.
Web app works	30	Typing a word and clicking the button shows the suggestions in a browser.
Deployed to a public URL	20	The app is live and working at a GitHub Pages (or equivalent) address.
Total	150	